IE 643: Deep Learning - Theory and Practice **July-Dec 2026**

Project Report: Birth-Death Accelerated Sampling for Score-Based Diffusion Mode

Team Name: TEAM NAME *Team Members: NAME (ROLL NO.), NAME (ROLL NO.)*

Abstract

Write here (own words): 4–6 sentences: the task (fewer diffusion sampling steps with birth-death), what goes wrong with few steps, your approach (birth-death after every noise level, density ratio estimated from the model), and the key numbers: e.g. on imbalanced MNIST, birth-death with half the steps (T=50) gives a better digit mix than the T=100 baseline (TV 0.062 vs 0.095) at similar FID (45.6 vs 49.7); at T=20 it halves the digit-mix error (0.091 vs 0.219); the label-free kNN estimator lowers FID from 49.7 to 30.1 at equal cost; on the 2D toy problem T=5 with birth-death beats T=100 without it.

1 Introduction

Write here (own words): Why diffusion sampling is slow (hundreds–thousands of network evaluations); why fewer steps fail (particles do not mix between modes; wrong mode proportions); the idea of birth-death (move mass by removing/duplicating particles instead of transporting it); why deep learning is needed (score network, classifier, denoiser). Last paragraph: the structure of the report (Sections 2–9).

2 Project Workflow

Write here (own words): Bullet points in the order you actually worked. Suggested list: (1) literature (NCSN, DDPM, DDIM, Lu et al.); (2) 2D toy: baseline annealed Langevin and step sweep; (3) toy: birth-death with exact potential; (4) toy: three estimators of V (energy network, heat-equation loss, ODE likelihood) and GPU timing; (5) MNIST: imbalanced data, noise-aware classifier, score network; (6) MNIST: baseline sweep, classifier and kNN birth-death; (7) ablations (step schedule, k, rate, V+KDE); (8) normal MNIST control; (9) interface and packaging. A workflow diagram (boxes and arrows) is recommended.

3 Literature Survey

Write here (own words): 2–3 paragraphs per paper, in your words: Song & Ermon [1] (noise-conditional score network, denoising score matching, annealed Langevin, low-density regions); Song & Ermon [2] (noise schedule, $s(x, \sigma) = s(x)/\sigma$, EMA); Ho et al. [4] (DDPM, U-Net with time embedding); Song et al. [5] (DDIM: fewer steps with a deterministic sampler – the standard acceleration approach, different from ours); Lu, Lu & Nolen [6] (birth-death Langevin, rate β, Algorithm 1, convergence independent of the energy barrier); Song et al. [3] (probability-flow ODE likelihood).

4 Proposed Approach

4.1 Background: score-based sampling

For $p_\sigma = p_{\text{data}} * \mathcal{N}(0, \sigma^2 I)$ with L=10 geometric noise levels $\sigma_1 > \cdots > \sigma_L$, a network $s_\theta(x, \sigma) \approx \nabla_x \log p_\sigma(x)$ is trained with denoising score matching [7, 1]:

$$\mathcal{L}(\theta) = \mathbb{E}_\sigma \, \mathbb{E}_{x \sim p_{\text{data}},\, z \sim \mathcal{N}(0,I)} \left[\tfrac{1}{2} \left\| \sigma \, s_\theta(x + \sigma z, \sigma) + z \right\|_2^2 \right]. \tag{1}$$

Annealed Langevin dynamics (NCSN) runs T steps at each level, from σ_1 down to σ_L:

$$x \leftarrow x + \eta_i \, s_\theta(x, \sigma_i) + \sqrt{2\eta_i}\, z, \qquad \eta_i = \epsilon \, (\sigma_i / \sigma_L)^2, \tag{2}$$

followed by one denoising step $x \leftarrow x + \sigma_L^2 s_\theta(x, \sigma_L)$. The cost is LT+1 network calls.

4.2 Birth-death step

Following Lu et al. [6], after the T Langevin steps of level i ($i < L$) we apply one birth-death step of length $\Delta t = T\eta_i$ (the Langevin time spent at that level) to the N particles:

$$\beta_j = \log \frac{\rho(x_j)}{\pi(x_j)} - \frac{1}{N} \sum_\ell \log \frac{\rho(x_\ell)}{\pi(x_\ell)}, \tag{3}$$

where ρ is the density of the particles and π the target (p_{σ_i}). Particle j is removed with probability $1 - e^{-\beta_j \Delta t}$ if $\beta_j > 0$ and duplicated with probability $1 - e^{\beta_j \Delta t}$ if $\beta_j < 0$; each event is paired with the opposite event on a random particle so N stays fixed.

Write here (own words): Explain in your words why this helps: Langevin moves particles locally and needs many steps to move mass between modes; birth-death moves mass between modes directly. Mention $\log \rho + V$ form in Lu et al., where $V = -\log \pi$.

Algorithm 1 Annealed Langevin with birth-death

$x_{1..N} \sim \mathcal{U}[0,1]^{784}$
for $i = 1, \ldots, L$ **do**
  for $t = 1, \ldots, T_i$ **do**
    Langevin step (2) on all particles
  if $i < L$ **then**
    estimate β_j (3) for all particles; remove / duplicate with $\Delta t = T_i \eta_i$
$x \leftarrow x + \sigma_L^2 s_\theta(x, \sigma_L)$

4.3 Work done before the prep-presentation review

Write here (own words): Fill in what you had at the prep review (e.g. literature study, toy baseline, birth-death with exact V on the toy problem, the three V estimators). Use the subsections below as material.

Estimating ρ/π on the 2D toy problem. (i) Exact $V = -\log p_\sigma$ with a Gaussian kernel density estimate of ρ (oracle). (ii) Energy network $E_\theta(x, \sigma)$ with $s = -\nabla_x E$ and $V = E$. (iii) Energy network with a heat-equation consistency loss: p_σ satisfies $\partial_\sigma p = \sigma \Delta p$, so with $\log p_\sigma = -E + c(\sigma)$

$$R(x, \sigma) = \frac{\partial E}{\partial \log \sigma} - \sigma^2 \big(\Delta_x E - \|\nabla_x E\|^2\big) = \sigma c'(\sigma) \quad \Rightarrow \quad \mathcal{L}_{\text{heat}} = \mathbb{E}_\sigma \operatorname{Var}_x R(x, \sigma). \tag{4}$$

(iv) Probability-flow ODE likelihood [3]: $V_{\sigma_0}(x_0) = -\log p_{\sigma_{\max}}(x(\sigma_{\max})) + \int_{\sigma_0}^{\sigma_{\max}} \sigma \nabla \cdot s(x(\sigma), \sigma)\, d\sigma$, $dx/d\sigma = -\sigma s(x, \sigma)$.

Write here (own words): Explain why (ii) fails at low noise (DSM only constrains ∇E where data has mass; the gap between modes is not determined) and why (iii) and (iv) fix it. See the methods note `estimating_V.pdf`.

4.4 Work done after the prep-presentation review

Why V + kernel density does not carry over to images.

Write here (own words): In 784 dimensions V of two equally good samples differs by about $\sqrt{784/2} \approx 20$ only because of the random norm of the noise; a kernel density from 1000 particles cannot cancel this, so β is dominated by noise. Cite your cell-20 test (Table 8).

Estimator A: noise-aware classifier. A classifier $p_\phi(k \mid x, \sigma)$ is trained on noisy training images. Writing the training distribution as $\pi = \sum_k \pi_k q_k$ and assuming the particles have the right shape within each digit but wrong digit masses, $\rho = \sum_k m_k q_k$, Bayes' rule gives for any target mix π'_k

$$\frac{\rho(x)}{\pi'(x)} = \frac{\sum_k (m_k/\pi_k)\, p_\phi(k \mid x, \sigma)}{\sum_k (\pi'_k/\pi_k)\, p_\phi(k \mid x, \sigma)}, \tag{5}$$

with the particle masses m_k estimated by label-shift EM [8]: iterate $r_{jk} \propto m_k p_\phi(k|x_j)/\pi_k$, $m_k = \frac{1}{N} \sum_j r_{jk}$.

Estimator B: k nearest neighbours in denoised space (label-free). Noisy training images $y + \sigma_i z$ are exact samples of p_{σ_i}. Particles and M=5000 such reference images are both mapped through the score model's denoiser $\phi(x) = x + \sigma^2 s_\theta(x, \sigma)$ (Tweedie), and nearest neighbours are found in denoised space, where images lie near the low-dimensional data manifold. This gives a *proxy* for ρ/π, not the exact ratio: for a map ϕ applied to both sets, the ratio of the pushed-forward densities at y equals $\mathbb{E}_\pi[\rho/\pi\,(x) \mid \phi(x) = y]$, i.e. the true ratio averaged over inputs with the same denoised image. It is exact only if ϕ is invertible (Jacobians cancel) or the ratio is constant along these sets; with a learned, approximate denoiser neither is guaranteed. The averaging removes variation along noise directions, which is what birth-death should ignore. If a_j of the k nearest neighbours of particle j are particles,

$$\log \frac{\rho}{\pi}(x_j) \approx \log \frac{a_j + \frac{1}{2}}{N} - \log \frac{k - a_j + \frac{1}{2}}{M}. \tag{6}$$

Write here (own words): Explain the two error sources (randomness of k counts vs. neighbourhood width) and that the k ablation shows robustness. Explain that estimator B also corrects the shape inside each digit, A only the digit masses.

5 Data set Details

Write here (own words): MNIST: source (torchvision / LeCun et al.), 60k/10k grayscale 28×28 images, 10 classes, pixels scaled to $[0, 1]$. Imbalanced version: keep 25% of digits 0–4 (seed 0) $\Rightarrow$ 37,050 training images, digits 0–4 4% each, 5–9 16% each (rare group 20%, as in the toy problem); test set built the same way (seed 1) is the FID reference. Normal MNIST: all images. Why imbalance: rare modes make mode-proportion errors measurable; real data is often long-tailed. Toy data: $0.2\,\mathcal{N}((-5,-5), I) + 0.8\,\mathcal{N}((5,5), I)$.

6 Experiments

Write here (own words): Hardware: Kaggle Tesla T4 16 GB. Code: GitHub link HERE. Training settings (Table 1). Metrics: TV (digit mix vs target, from hard predictions of the evaluation classifier), classifier-feature FID (FID formula on 128-d evaluation-classifier features; not comparable to Inception FID; reference = imbalanced test set), confidence, near-duplicates ($L_2 < 1$), score-network calls, wall-clock time. 1000 samples per run, 3 seeds (mean $\pm$ std). All samplers use the same trained networks and the same initial noise. Include ALL experiments, also negative ones (step schedule, V+KDE, T=5).

Table 1: Training and sampling settings.

Component	Architecture	Training
Score network	U-Net 32/64/128 ch., GroupNorm, SiLU, σ embedding, output/σ, 1.0M params	DSM, 40k iters, batch 128, Adam 2×10^{-4}, EMA 0.999
Noise-aware classifier	3 conv layers with FiLM(σ)	CE, 6k iters, batch 256, Adam 10^{-3}, cosine
Evaluation classifier	4 conv + FC 128 (clean, balanced MNIST)	CE, 4 epochs, augmentation; 98.76% test acc.
Noise levels	L=10, geometric, σ_1=14.9 (max pairwise distance), σ_L=0.01	–
Sampler	$\epsilon = 2\times10^{-5}$, $T \in \{100, 50, 20, 10, 5\}$, final denoising step	1000 samples, seeds 0–2

Table 2: Noise-aware classifier on the imbalanced test set: accuracy and calibration error (ECE) at each noise level. At the three highest levels the rare digits 0–4 have 0% accuracy (the classifier falls back to the training mix).

σ	14.9	6.62	2.94	1.31	0.579	0.257	0.114	0.051	0.023	0.010
accuracy	0.177	0.191	0.248	0.496	0.909	0.976	0.984	0.984	0.984	0.985
ECE	0.006	0.007	0.017	0.011	0.008	0.008	0.008	0.009	0.009	0.007

7 Results

7.1 2D toy problem

Table 3: Toy problem, split error (target 20/80), mean of 3 seeds; SW distance in brackets.

T	Baseline	BD, exact V	BD, classifier	BD, kNN denoised
100	3.2% (0.39)	0.6% (0.20)	2.5% (0.33)	1.2% (0.10)
20	2.9% (0.33)	0.9% (0.21)	0.6% (0.24)	0.8% (0.13)
10	8.3% (0.93)	0.7% (0.23)	0.2% (0.54)	1.8% (0.27)
5	15.2% (2.58)	0.7% (0.32)	0.5% (1.55)	2.9% (0.48)

Table 4: Toy problem, mode gap $V(\mu_L) - V(\mu_R)$ (true 1.39) for two training runs, and GPU benchmark (T4, N=1000).

σ	energy E	E+heat	ODE	Setup (T)	time	speed-up	split err
6.16	1.17 / 1.05	1.10 / 1.32	1.27 / 1.33	baseline (100)	0.338 s	1.0×	4.7%
1.90	1.65 / 1.05	1.32 / 1.59	1.52 / 1.65	BD exact V (5)	0.035 s	9.8×	1.4%
0.58	2.07 / 1.56	1.28 / 1.53	1.57 / 1.75	BD E+heat (5)	0.073 s	4.6×	0.2%
0.10	4.63 / 3.50	1.27 / 1.55	1.59 / 1.75	BD ODE (5)	0.177 s	1.9×	1.0%

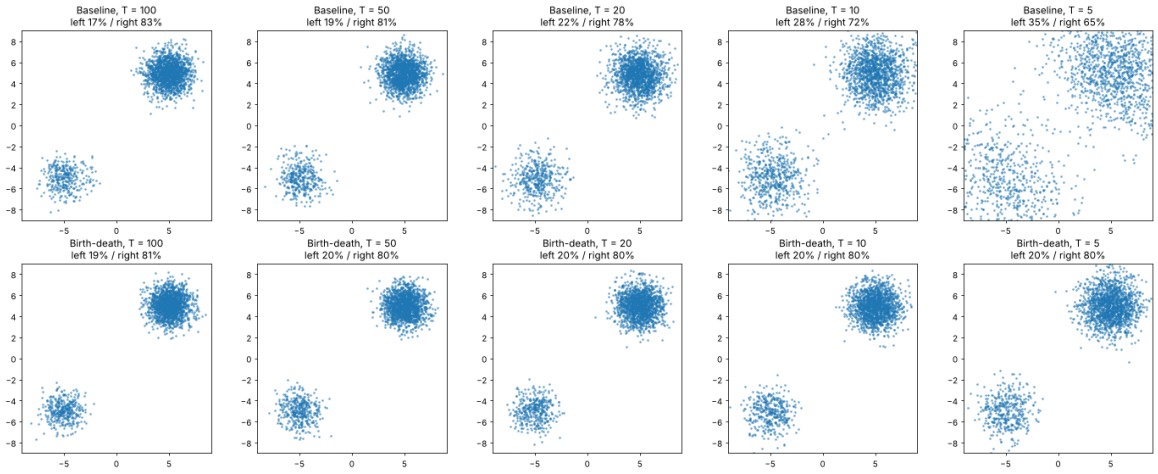


Figure: toy samples, baseline (top) vs birth-death with exact V (bottom).

Write here (own words): Interpret: fewer steps break the split; birth-death fixes it; plain energy fails at low noise, heat loss and ODE fix it; results vary between training runs (state this).

7.2 MNIST (imbalanced)

missing: `figures/samples_all.png`

Figure: samples (baseline T=100, baseline T=10, BD classifier T=10, BD kNN T=10).

missing: `figures/quality_vs_calls_all.png`

Figure: TV and FID vs score-network calls.

Table 5: Imbalanced MNIST, 1000 samples, mean ± std over 3 seeds. FID = classifier-feature FID. Real-data reference: 1000 held-out real training images vs the test-set reference, TV 0.055, FID 21.40 (two disjoint real test subsets: FID 20.24). Target rare share 0.206. Score calls include the 9 extra denoising calls of kNN; the classifier estimator adds 9 classifier calls (not score calls). The baseline T=100 time has std 5.7 s (first run); all other runtime std are below 1.5 s.

Method	T	score calls	time (s)	vs base T=100	vs base same T	TV	FID	near-dup
baseline	100	1001	99.7	1.0×	1.00×	0.095 ± 0.011	49.7 ± 5.3	0.000
BD classifier	100	1001	106.3	0.9×	0.94×	0.088 ± 0.009	51.7 ± 11.2	0.003
BD kNN	100	1010	108.6	0.9×	0.92×	0.050 ± 0.003	30.1 ± 3.6	0.006
baseline	50	501	53.7	1.9×	1.00×	0.109 ± 0.016	68.3 ± 11.0	0.000
BD classifier	50	501	53.7	1.9×	1.00×	0.068 ± 0.006	57.4 ± 2.6	0.001
BD kNN	50	510	54.5	1.8×	0.98×	0.062 ± 0.015	45.6 ± 7.0	0.003
baseline	20	201	21.6	4.6×	1.00×	0.219 ± 0.003	202 ± 16	0.000
BD classifier	20	201	21.6	4.6×	1.00×	0.116 ± 0.013	159 ± 6	0.001
BD kNN	20	210	22.5	4.4×	0.96×	0.091 ± 0.009	117 ± 8	0.005
baseline	10	101	10.9	9.2×	1.00×	0.333 ± 0.018	473 ± 28	0.000
BD classifier	10	101	10.9	9.1×	0.99×	0.227 ± 0.013	419 ± 6	0.002
BD kNN	10	110	11.9	8.4×	0.92×	0.243 ± 0.027	388 ± 20	0.004
baseline	5	51	5.5	18.2×	1.00×	0.734 ± 0.002	1793 ± 4	0.000
BD classifier	5	51	5.6	17.9×	0.98×	0.732 ± 0.003	1762 ± 6	0.001
BD kNN	5	60	6.5	15.4×	0.85×	0.729 ± 0.003	1784 ± 9	0.002

missing: `figures/digit_proportions.png`

Figure: digit proportions vs target.

missing: `figures/bd_per_level.png`

Figure: estimated rare share and birth/death events per noise level.

Write here (own words): Interpret honestly: at T=50 (half the steps) both BD versions have a better digit mix than baseline T=100 (TV 0.062 / 0.068 vs 0.095), and kNN also has a slightly lower FID (45.6 vs 49.7, within one std); at T=20 BD roughly halves TV (0.091 / 0.116 vs 0.219) and cuts FID from 202 to 117 (kNN); at T=100 kNN improves FID from 49.7 to 30.1 at equal cost; T=10 still clearly better than baseline but far from T=100; T=5 collapses for all methods (birth-death can only select among the samples Langevin produces). TV and FID measure different things and neither proves the full distribution is preserved. Near-duplicates stay $\leq 0.6\%$. Overhead: classifier about +0.1 s, kNN about +1 s per run of 1000 samples (the +7 s at T=100 is timing noise of the first baseline run).

Table 6: Left: step schedule at a fixed budget of 100 Langevin steps, FID / TV (mean of 2 seeds). Right: number of neighbours k for kNN birth-death at T=20 (baseline: TV 0.219, FID 202).

Schedule	baseline	BD clf	BD kNN
uniform (10 each)	469 / 0.323	421 / 0.230	384 / 0.237
fewer at high noise	724 / 0.405	660 / 0.362	656 / 0.392
more at high noise	466 / 0.292	483 / 0.226	436 / 0.214

k	TV	FID	near-dup
5	0.124	139	0.003
10	0.129	140	0.012
20	0.086	117	0.007
50	0.101	121	0.007
100	0.136	142	0.005

7.3 Ablations

Write here (own words): Step schedule: cutting high-noise steps is worst – the digit mix is decided at high noise and the classifier is blind there (Table 2). More steps at high noise helps the baseline and kNN slightly. k: every value beats the baseline; k=20–50 is best.

Table 7: Birth-death rate multiplier (mean of 2 seeds), TV / FID. Near-duplicate share in brackets when above 1%.

Estimator	T	rate 1	rate 3	rate 10
classifier	20	0.109 / 156	0.095 / 160	0.142 / 213 (1.4%)
classifier	10	0.230 / 421	0.211 / 418	0.174 / 411
kNN	20	0.086 / 117	0.093 / 118	0.078 / 107 (3.5%)
kNN	10	0.237 / 384	0.199 / 371	0.201 / 348 (2.6%)

Write here (own words): Rate: a stronger rate helps a little at T=10 but creates more duplicates; at T=20 rate 10 hurts the classifier version. Rate 1 (the Lu et al. value) is a safe default.

Table 8: Original Lu et al. rule on MNIST: V from the probability-flow ODE (4 Hutchinson probes) + kernel density. Test A: spread of β and share of particles killed or duplicated in one birth-death step, compared with kNN. Test B, full sampler at T=20: seed 0 TV 0.158, FID 177, near-dup 44.2%; seed 1 TV 0.115, FID 136, near-dup 47.3%; 40 s per run (kNN: TV 0.091, FID 117, near-dup 0.5%, 22.5 s).

σ	std(β) V+KDE	acted on	std(β) kNN	acted on
3.00	20.2	95.9%	0.62	34.1%
1.33	21.6	96.3%	0.72	37.5%
0.588	21.0	96.7%	0.73	37.9%
0.260	25.2	97.2%	0.72	37.4%
0.115	31.0	98.0%	0.72	37.6%

Write here (own words): V+KDE: β spreads by about 20 $\approx \sqrt{d/2}$ (expected fluctuation of $-\log p$ in d=784), so almost every particle is killed or duplicated at every level and about half the samples become copies; 2× slower than kNN. Negative result: the original rule does not work directly on images.

7.4 Normal (balanced) MNIST

Write here (own words): Table with the same columns as Table 5 from the balanced run (cells 12 and 16). Interpret whether birth-death still helps / does not create an imbalance.

7.5 Interface

Write here (own words): Describe `app.py` (Gradio): inputs (model, estimator, T, number of samples, rate, k, target digit mix, seed), outputs (two sample grids from the same initial noise, digit-proportion plot, metrics table). How to run (`python app.py --model NAME:DIR:SIGMA_MAX:KEEP --share`). Add a screenshot.

8 Plan for Novelty Assessment

Write here (own words): Ideas: (1) potential-based birth-death on images with V from the ODE / heat-loss energy model computed in a learned low-dimensional feature space; (2) heat-equation-regularised energy network on MNIST; (3) birth-death rate that adapts to T (stronger at small T) and to the classifier's confidence; (4) combine birth-death with faster samplers (DDIM / probability-flow ODE); (5) CIFAR-10; (6) learned metric for the kNN estimator.

9 Conclusion

Write here (own words): Summary: problem, methods (oracle on toy, three V estimators, classifier and kNN estimators on MNIST), main results with numbers, limitations (labels for the classifier estimator; FID gap at reduced steps; failure at T=5; V estimation in high dimension; variation between training runs), future work.

References

[1] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution, Advances in Neural Information Processing Systems (NeurIPS), 2019. `https://arxiv.org/abs/1907.05600`

[2] Yang Song and Stefano Ermon. Improved techniques for training score-based generative models, Advances in Neural Information Processing Systems (NeurIPS), 2020. `https://arxiv.org/abs/2006.09011`

[3] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations, International Conference on Learning Representations (ICLR), 2021. `https://arxiv.org/abs/2011.13456`

[4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, Advances in Neural Information Processing Systems (NeurIPS), 2020. `https://arxiv.org/abs/2006.11239`

[5] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models, International Conference on Learning Representations (ICLR), 2021. `https://arxiv.org/abs/2010.02502`

[6] Yulong Lu, Jianfeng Lu, and James Nolen. Accelerating Langevin sampling with birth-death, arXiv preprint, 2019. `https://arxiv.org/abs/1905.09863`

[7] Pascal Vincent. A connection between score matching and denoising autoencoders, Neural Computation 23(7), 2011. `https://doi.org/10.1162/NECO_a_00142`

[8] Marco Saerens, Patrice Latinne, and Christine Decaestecker. Adjusting the outputs of a classifier to new a priori probabilities: a simple procedure, Neural Computation 14(1), 2002. https://doi.org/10.1162/089976602753284446

[9] Yang Song and Stefano Ermon. https://github.com/ermongroup/ncsn, 2019. Accessed on: 12th October, 2026.

[10] Yang Song and Stefano Ermon. https://github.com/ermongroup/ncsnv2, 2020. Accessed on: 25th October, 2026.